

TransSGPN: Transistor Network Synthesis via Switching Graph Policy Network

Supplementary Material

Anonymous submission

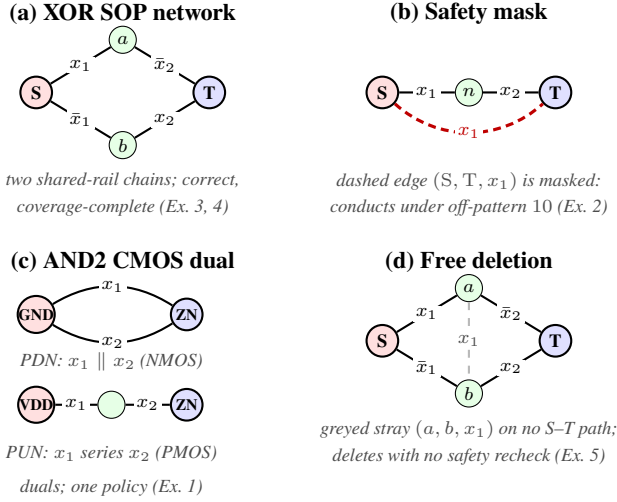


Figure 1: Worked 2-variable circuits for the theoretical results. (a) The XOR SOP network (Example 3): two shared-rail chains, correct and coverage-complete, reachable in any insertion order (Example 4). (b) A safe series chain for x_1x_2 ; the dashed shortcut (S, T, x_1) is rejected by the safety mask because it conducts under the off-pattern 10 (Example 2). (c) The AND2 cell: parallel NMOS pull-down, series PMOS pull-up; duals synthesized by one policy (Example 1). (d) A structurally dead transistor (grey), free to delete with no off-pattern recheck (Example 5). Circles S/T are the source/sink rails; small green nodes are internal.

This supplement collects worked examples, full proofs, implementation details, and the training pipeline referenced in the main paper. Proposition, theorem, corollary, and lemma numbers match those of the main paper; figure, table, and example numbers are local to this supplement.

A Worked Examples

Example 1 (Decomposing an AND2 cell). Let $f = x_1x_2$, so $P^+ = \{11\}$ and $P^- = \{00, 01, 10\}$ (Figure 1(c)). The CMOS cell splits into two independent switching problems. The PDN realizes $\neg f$ with on-patterns P^- : it conducts whenever $x_1=0$ or $x_2=0$, the *parallel* pair $x_1 \parallel x_2$ between

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

GND and ZN. The PUN realizes $f(\neg x)$ with on-patterns $\{\neg 11\} = \{00\}$: it conducts only when $x_1=0$ and $x_2=0$, the *series* pair between VDD and ZN. The two networks are duals (series $\leftrightarrow$ parallel), synthesized by the *same* policy conditioned on their respective pattern sets.

Example 2 (Monotonicity and incremental safety). Take $f = x_1x_2$, so $P^- = \{00, 01, 10\}$. Suppose the partial network already contains the series pair (Figure 1(b)) $\text{SRC} \xrightarrow{x_1} n \xrightarrow{x_2} \text{SNK}$. This network is *safe*: under 00, 01, 10 at least one of x_1, x_2 is 0, so no $\text{SRC} \rightarrow \text{SNK}$ path conducts. Now add the shortcut $e = (\text{SRC}, \text{SNK}, x_1)$. By monotonicity (Theorem 2) every previously conducting path still conducts, so we test only whether e *by itself* creates an off-pattern path (Corollary 3): under $10 \in P^-$, $x_1 = 1$, so e alone connects SRC to SNK and conducts illegally, and e is masked out. Adding $e' = (\text{SRC}, \text{SNK}, x_1x_2)$ would be admissible, since $x_1x_2 = 0$ on all of P^- . We reach this verdict by checking only the new edge against $|P^-| = 3$ patterns, never re-examining the existing chain.

Example 3 (SOP network for a 2-variable function). For $g = x_1x_2 + \bar{x}_1x_2$ (XOR), $P^+ = \{10, 01\}$. $G_{\text{SOP}}(g)$ places one 2-transistor chain per on-pattern between the shared rails (Figure 1(a)): chain A is $\text{SRC} \xrightarrow{x_1} a \xrightarrow{\bar{x}_2} \text{SNK}$ and chain B is $\text{SRC} \xrightarrow{\bar{x}_1} b \xrightarrow{x_2} \text{SNK}$. Under 10, chain A conducts and $g = 1$; under 01, chain B conducts. Under 00, 11 each chain has one broken transistor, so nothing conducts. This 4-transistor network is correct but not minimal, since XOR admits a 3-transistor bridge. It serves as the fixed *compensate* network C_g that conditions the state.

Example 4 (The mask never blocks a valid target). Let G^* be the XOR network of Example 3 with edges $\{e_1, e_2\}$ (chain A), $\{e_3, e_4\}$ (chain B), built in the interleaved order e_1, e_3, e_2, e_4 that never completes a chain until the end. At each prefix the partial network is a subgraph of G^* ; by monotonicity any off-pattern path in a prefix would also exist in the safe G^* , so no prefix conducts illegally and the mask rejects none of the four placements, as for all $4! = 24$ orderings. Thus, although path-order supervision presents one linearization, completeness guarantees the policy is free to discover G^* in any order.

Example 5 (A redundant transistor is free to delete). Suppose the policy synthesizes the XOR pull-down of Example 3

(chains A and B) but also emits a stray device $s = (a, b, x_1)$ linking the two internal nodes (Figure 1(d)). Deleting s yields $G' \subseteq G$; by Lemma 4 no off-pattern check is needed, so we re-verify only coverage: chains A and B still carry 10 and 01, hence G' computes the same function. Because s lies on no $\text{SRC} \rightarrow \text{SNK}$ path it is structurally dead, and the reported cost is $T_{\text{eff}} = 4$ rather than 5. On the 6-input NSP benchmark this accounting difference is the gap between a mean of 41.4 and 19.0 transistors.

B Proofs

Results are restated in the order they appear in the main paper, and the numbering below matches it.

Proposition 1 (Dual Switching Network Equivalence). *Let P^+ and P^- denote the on- and off-patterns of the cell function f . A CMOS cell correctly implements $f(\mathbf{x})$ if and only if its PDN correctly implements $\neg f(\mathbf{x})$, with on-patterns P^- , and its PUN correctly implements $f(\neg \mathbf{x})$, with on-patterns $\{\neg \pi : \pi \in P^+\}$.*

Proof. (PDN.) An NMOS transistor with gate literal x_i conducts exactly when $x_i = 1$, so its behaviour coincides with the switch semantics of Definition 1 with no relabelling. By definition the cell is correct only if the PDN pulls the output low whenever $f(\mathbf{x}) = 0$, that is, only if it conducts for every $\pi \in P^-$, and only if it leaves the output floating whenever $f(\mathbf{x}) = 1$, that is, only if it conducts for no $\pi \in P^+$. These are precisely coverage and safety for the function $\neg f$, whose on-patterns are P^- .

(PUN.) A PMOS transistor with gate literal x_i conducts exactly when $x_i = 0$, so it behaves as a switch controlled by $\neg x_i$. Substituting $\mathbf{y} = \neg \mathbf{x}$ turns every PMOS gate into a positive literal in $\mathbf{y}$, so the PUN, read as a switching network over $\mathbf{y}$, conducts iff $f(\neg \mathbf{y}) = 1$. The cell is correct only if the PUN pulls the output high whenever $f(\mathbf{x}) = 1$, and by the substitution this is the condition $\mathbf{y} \in \{\neg \pi : \pi \in P^+\}$. Hence the PUN implements $f(\neg \mathbf{x})$ with that on-pattern set.

Both directions of the equivalence follow, since each step above is a restatement of the cell-correctness condition of Definition 2 rather than an implication in one direction only. $\square$

Theorem 2 (Monotonicity of Conducting Paths). *For any transistor network G and any transistor $e \notin E(G)$, $\text{Paths}(G) \subseteq \text{Paths}(G \cup \{e\})$.*

Proof. Adding e can only activate new conducting channels; it cannot deactivate any already-conducting transistor. Hence every path that conducted in G still conducts in $G \cup \{e\}$. $\square$

Corollary 3 (Incremental Safety Checking). *If G is safe for g , then $G \cup \{(u, v, \ell)\}$ is safe for g iff (u, v, ℓ) alone creates no $\text{SRC} \rightarrow \text{SNK}$ path under any $\pi \in P^-$.*

Proof. Any P^- -path present in $G \cup \{e\}$ must use e , since G was safe. Checking e alone therefore suffices. $\square$

Lemma 4 (Deletion Preserves Safety). *If $G' \subseteq G$ and G is safe for g , then G' is safe for g .*

Proof. Every path in G' is a path in G , so a violating path in G' would also violate G . $\square$

Theorem 5 (SOP Network Correctness). *Let $G_{\text{SOP}}(g)$ be the series-parallel network with one N -transistor series chain per $\pi \in P^+$, all sharing SRC and SNK . Then $G_{\text{SOP}}(g)$ correctly implements g .*

Proof. Coverage: for $\pi \in P^+$ every literal in the chain built for π evaluates to 1, forming a complete $\text{SRC} \rightarrow \text{SNK}$ path. Safety: for $\pi' \in P^-$ and any chain built for $\pi \in P^+$, since $\pi' \neq \pi$ they differ at some bit j^* , so the j^* -th transistor of that chain does not conduct under π' . Every chain is therefore broken. $\square$

Theorem 6 (Completeness of Safety-Masked Search). *For any valid G^* implementing g , every permutation of $E(G^*)$ is a valid trajectory under the safety mask.*

Proof. Let $e_1, \dots, e_T$ be any ordering of $E(G^*)$ and $G_t = \{e_1, \dots, e_t\}$. Base: $G_0 = \emptyset$ is safe. Step: suppose G_{t-1} is safe. Since $G_t \subseteq G^*$, any P^- -path in G_t would persist in G^* by monotonicity, contradicting the safety of G^* . Hence G_t is safe and e_t is never rejected. $\square$

C Implementation Details

C.1 Placement Metrics

Placement arranges the cell's transistors into two rows sharing a common gate column structure: NMOS devices (PDN) in the bottom row, PMOS devices (PUN) in the top row, both connected to the output node ZN and separated by routing tracks. Transistors adjacent in a row can share a diffusion region when their abutting terminals carry the same net, so the ordering of devices within each row, and not only their number, determines the area the cell occupies. The five metrics entering Φ are:

- **Cell width** W_{cpp} : the number of poly columns the cell spans, in *contacted poly pitch* (CPP) units.
- **Gate mismatches** N_{gm} : column positions at which the P-row and N-row devices are driven by different gate signals, preventing a shared poly strip.
- **Wire length** W_{wl} : total intra-cell routing length.
- **Routing density** D_{rt} : peak track congestion, which bounds routability.
- **Diffusion cuts** N_{gap} : breaks in the row's diffusion, required wherever consecutive devices cannot abut. A row admits a cut-free placement exactly when its device graph has an Euler path, so N_{gap} measures departure from that ideal.

C.2 SPICE Generation

A complete PDN+PUN pair is encoded as a standard SPICE .subckt. NMOS transistors (NMOS_VTL, $W = 90$ nm, $L = 50$ nm) implement the PDN with source at GND, drain at the output ZN, and internal nodes named ndk ; PMOS transistors implement the PUN symmetrically between VCC and ZN. Gate signals map directly from literal labels: a positive literal x_i becomes port x_i , and a negative literal $\neg x_i$

becomes the complemented port x_{i_N} . Power nets are named VCC/GND to match ASTRAN’s default supply identifiers.

C.3 ASTRAN Placement

ASTRAN places the combined netlist using Threshold Accepting with parameters fold 2 0, SA quality 1, and 1 at-tempt, and reports the five placement metrics of App. C.1 on its “Final cost” line. The weights in Φ are left at the tool’s own defaults, so minimizing Φ coincides exactly with minimizing ASTRAN’s internal SA objective. Calls are dispatched over a thread pool, one process per complete pair.

C.4 Hyperparameters

Table 1: Architecture and training hyperparameters. ASTRAN place weights are listed in the order $(w_c, w_m, w_w, w_d, w_g)$; the edge feature dimension is 5/6/8 for $N=3/4/6$.

Hyperparameter	Value
MPNN hidden dim d	128
MPNN layers L	3
Head MLP hidden dim	256
Post-MPNN self-attention	4 heads
Node feature dim d_{node}	4
Edge feature dim d_{edge}	$N + 2$
Phase 1 lr (cosine)	$2 \times 10^{-3} \rightarrow 10^{-4}$
Phase 1 batch size	512
Phase 1 epochs	200
Head-1 loss weight	2
Phase 2 lr	10^{-4}
Phase 2 trajectories K	16
PPO clip ε	0.2
Entropy coeff λ_H	0.05
Adaptive-anchor β	0.3
Restarts	2
Max updates per target	200
Max steps T_{max}	20
Phase 3 lr	3×10^{-5}
Phase 3 pairs K	8
ASTRAN SA quality	1
ASTRAN place weights	(3, 4, 1, 4, 2)
ASTRAN parallel workers	4

D Training Pipeline

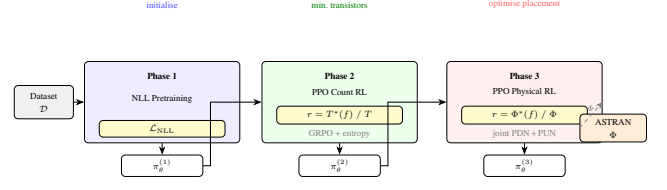


Figure 2: Three-phase training pipeline of TransSGPN. Phase 1 initialises the SGPN by NLL imitation on known-optimal circuits. Phase 2 finetunes toward minimum transistor count via PPO with adaptive reward $r = T^*(g)/T_{\text{eff}}$ and a GRPO baseline. Phase 3 jointly synthesises PDN and PUN and uses the ASTRAN placement objective Φ directly as the reward.

Algorithm 1 TransSGPN / SGPN Training (Phases 1–2)

Require: Dataset $\mathcal{D}$, target list $\mathcal{F}$, hyperparameters $K, M, \lambda_H, \varepsilon_{\text{clip}}$

- 1: Pretrain policy π_θ by NLL on $\mathcal{D}$ (Phase 1)
- 2: Initialize frozen agent $\pi_{\text{old}} \leftarrow \pi_\theta$
- 3: **for** each epoch **do**
- 4: Sample batch of targets $\mathcal{B} \subseteq \mathcal{F}$
- 5: **for** each $g \in \mathcal{B}$ **do**
- 6: Generate K trajectories using π_{old}
- 7: Reduce each complete network to T_{eff}
- 8: Update $T^*(g)$ if a shorter complete network is found
- 9: Compute rewards $\{r_k\}$ and GRPO advantages $\{\hat{A}_k\}$
- 10: Compute batched PPO + entropy loss
- 11: Update θ via Adam
- 12: **if** step $\equiv 0 \pmod{M}$ **then** $\pi_{\text{old}} \leftarrow \pi_\theta$

Algorithm 2 TransSGPN Phase 3: Physical-Aware Joint PDN+PUN RL

Require: Phase 2 policy π_θ , function list $\mathcal{F}$, ASTRAN binary, weights $(w_c, w_m, w_w, w_d, w_g)$

- 1: Initialize $\Phi^*(f) \leftarrow \infty$ for all f
- 2: $\pi_{\text{old}} \leftarrow \pi_\theta$
- 3: **for** each epoch **do**
- 4: Sample batch $\mathcal{B} \subseteq \mathcal{F}$
- 5: **for** each $f \in \mathcal{B}$ **do**
- 6: **for** $k = 1, \dots, K$ **do**
- 7: $\tau_k^{\text{PDN}} \leftarrow \text{generate}(\pi_{\text{old}}, \neg f(\mathbf{x}))$
- 8: $\tau_k^{\text{PUN}} \leftarrow \text{generate}(\pi_{\text{old}}, f(\neg \mathbf{x}))$
- 9: **if** both complete **then**
- 10: $\Phi_k \leftarrow \text{ASTRAN}(\text{SPICE}(\tau_k^{\text{PDN}}, \tau_k^{\text{PUN}}))$
- 11: Update $\Phi^*(f)$ if $\Phi_k < \Phi^*(f)$
- 12: $r_k \leftarrow \Phi^*(f)/\Phi_k$
- 13: **else** $r_k \leftarrow 0$
- 14: GRPO: $\hat{A}_k \leftarrow (r_k - \bar{r})/(\sigma_r + \varepsilon)$
- 15: Update θ via PPO + entropy loss on all steps
- 16: **if** step $\equiv 0 \pmod{M}$ **then** $\pi_{\text{old}} \leftarrow \pi_\theta$